PHISHGUARD

A fine-tuned NVIDIA Nemotron that catches scams on every channel

Fine-tuning: 90.7% → 97.8%

Beats the frontier model Kimi K3 (96.2%) on 600 held-out cases · about 7× cheaper

Team: John Shi · Kris Chai | github.com/yuansshi/fireworks_hackthon

1 · THE PROBLEM

Scams hit every channel. Screening them all is too expensive.

4

channels to screen:
email, SMS, links, calls

$1,600

per 1M messages with a
frontier model (Kimi K3)

Now

a verdict is needed before
the click or the call ends

Who it is for: fraud and security teams at banks, telcos and enterprises that must screen every message, fast, without sending customer data to a third party.

2 · TECH STACK

What the stack made possible

Fireworks Serverless Training	LoRA fine-tune of a 120B model in ~11 min for ~$2.20: no GPUs, no dataset upload, no deployment
Fireworks serverless inference	Frontier baselines (Kimi K3 2.8T, DeepSeek V4.1 Flash) through one API, JSON-schema output
NVIDIA Nemotron 3.5 Super	Open 120B MoE (12B active): strong base, self-hostable for private screening
Hugging Face + Python	5 public datasets across 4 channels; async eval harness, bootstrap CIs, McNemar tests

Impossible without it: tuning a 120B model with no GPUs in minutes, then scoring it against a 2.8T frontier model through the same API.

2 · TECH STACK: THE EVALUATION

One evaluation, frozen before any run

2,400
training examples
(600 per channel)

600
held-out test cases
(150 per channel)

4
systems, same prompt,
same scoring

Kimi K3 (frontier) · DeepSeek V4.1 Flash · Nemotron 3.5 Super untuned (**before**) · Nemotron 3.5 Super + LoRA (**after**)

Plus 200 later-corpus emails, 200 recent synthetic scams and 300 call prefixes for drift and early warning

3 · LIVE DEMO

Live demo: held-out messages

```
$ uv run python scripts/demo.py --models ft-nemotron-super kimi-k3

[SMS]  truth: MALICIOUS
  Here is your discount code RP176781. To stop further messages reply stop…
  ✔ ours    MALICIOUS    recorded    $156 / 1M msgs
  ✔ Kimi K3 MALICIOUS    1.33 s      $1,218 / 1M msgs

[EMAIL]  truth: benign
  Subject: RE: LOCATION FOR MAN NIGHT  Either way, it sounds like…
  ✔ ours    benign       recorded    $518 / 1M msgs
  ✔ Kimi K3 benign       0.91 s      $2,973 / 1M msgs

[URL]  truth: MALICIOUS
  https://www.citieid.com/kenya
  ✘ ours    benign       recorded    $153 / 1M msgs
  ✘ Kimi K3 benign       0.69 s      $1,212 / 1M msgs

[CALL, first 50%]  truth: MALICIOUS
  caller: Hello, this is Karen from the Microsoft Refund Department…
  ✔ ours    MALICIOUS    recorded    $275 / 1M msgs
  ✔ Kimi K3 MALICIOUS    1.45 s      $1,836 / 1M msgs
```

Ours: verdicts recorded in the evaluation run (adapter sampled in its training session) · Kimi K3: live API calls

3 · CODE: FINE-TUNING

The training loop

```
svc = FiretitanServiceClient(api_key=KEY, base_url="https://api.fireworks.ai/training/v1/serverless")
tc = svc.create_lora_training_client(base_model=NEMOTRON_SUPER_120B, rank=16)

def to_datum(tok, row):                        # train only on the answer tokens
    prompt = render(tok, row["messages"][:-1], add_generation_prompt=True)
    full = render(tok, row["messages"], add_generation_prompt=False)
    w = [0.0] * len(prompt) + [1.0] * (len(full) - len(prompt))
    return Datum(model_input=ModelInput.from_ints(full[:-1]),
                 loss_fn_inputs={"target_tokens": full[1:], "weights": w[1:]})

for batch in batches(datums, size=32):         # 75 steps = 1 epoch, ~11 min
    tc.forward_backward(batch, "cross_entropy")
    tc.optim_step(AdamParams(learning_rate=1e-4)).result()
ckpt = tc.save_weights_for_sampler("phish-ep1").result().path   # sampled in-session: no deploy
```

Excerpt, simplified from scripts/train_serverless.py

3 · CODE: FEATURES AND SCORING

Metadata in, label out, paired scoring

```
meta = {**link_summary(text),                  # domains, shorteners, IP hosts, punycode
        **content_cues(text),                  # urgency words, deadlines, OTP / gift-card asks
        **time_features(sent_at)}              # send hour, off-hours
user = f"Channel: {channel}\nMetadata:\n{format_meta(meta)}\nContent:\n{text}"
# every model returns {"label": "malicious" | "benign"} (JSON schema, temperature 0)

# same 600 frozen cases for every system, then a paired McNemar test
b = sum(ours_ok & ~kimi_ok)                    # only ours right: 15
c = sum(kimi_ok & ~ours_ok)                    # only Kimi K3 right: 5
p = binomtest(b, b + c, 0.5).pvalue            # p = 0.041
```

Excerpts from src/phish/features.py, prompt.py and report.py

RESULTS: BEFORE VS AFTER FINE-TUNING

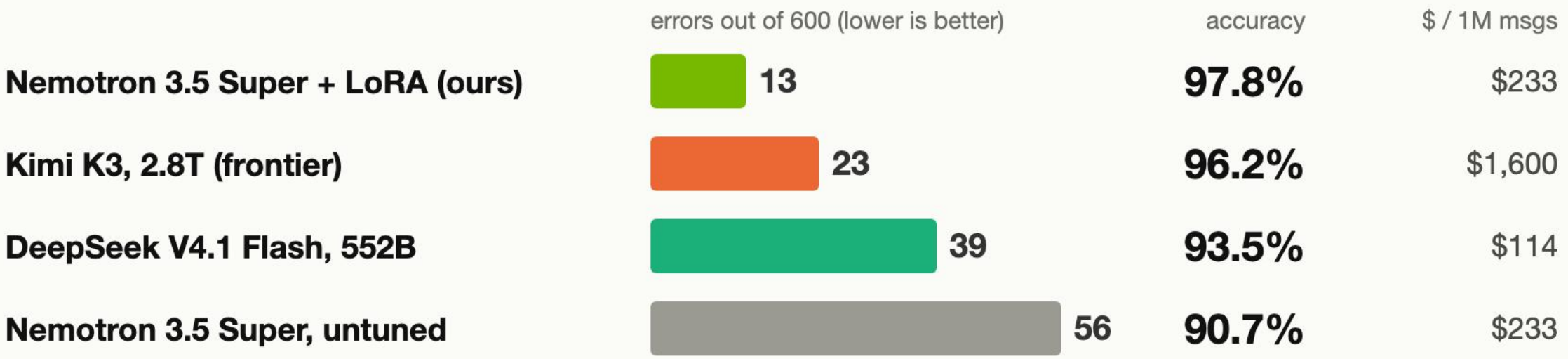
Same model, same prompt: before vs after fine-tuning

90.7% → 97.8% accuracy on 600 held-out cases

Nemotron 3.5 Super	before	after
Errors out of 600	56	**13 (–77%)**
False-positive rate	15.0%	**1.0%**
Accuracy by channel: email · SMS · URL · calls	94 · 94 · 92 · 83	**97 · 97 · 97 · 100**
Call false alarms after hearing 25% of the call	34%	**2%**
Later, unseen email corpus (regression)	87.0%	**85.5%**

RESULTS: VS THE FRONTIER

After fine-tuning, it beats the frontier



13 vs 23 errors: McNemar p = 0.041 · ΔF1 +1.7 [+0.2, +3.3] · about 7× cheaper than Kimi K3

Tradeoff: Kimi K3 generalizes better to newer, unseen data (2008 corpus F1 91.5 vs 83.8)

WHY ADOPT

PhishGuard

+7.1

accuracy points from fine-tuning (90.7 → 97.8)

~7×

cheaper per message than Kimi K3

23×

smaller model (120B vs 2.8T)

An open NVIDIA Nemotron model you can self-host, so customer messages stay on your infrastructure.

Reproduce everything: **github.com/yuansshi/fireworks_hackthon**